VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY

PROJECT REPORT

Course: Fundamentals of Programming

MUMMY MAZE

Class: 25C11
Group: 06
Instructor:
PhD. Lê Thanh Tùng
Mr. Trần Hoàng Quân
Mr. Nguyễn Thanh Tình
Member:
25127012 – Dương Trung Anh
25127013 – Hà Trần Bội Anh
25127101 – Nguyễn Tuấn Nghĩa
25127131 – Nguyễn Nhật Quỳnh
25127431 – Trần Ngọc Nghĩa

HO CHI MINH CITY, DECEMBER 2025

INTRODUCTION

This project is conducted as part of the *Fundamentals of Programming* course and focuses on the development of **Maze of Malice**, a 2D puzzle game inspired by the classic *Mummy Maze*. Rather than directly replicating the original game, our group adapts its core gameplay mechanics into a dungeon-themed environment, where the player assumes the role of an adventurer attempting to escape from a maze while avoiding various enemy entities and environmental hazards.

Maze of Malice is designed around a turn-based movement system, emphasizing strategic planning and logical decision-making. Each level presents the player with a maze of increasing complexity, requiring careful consideration of movement choices to avoid enemies, traps, and other obstacles while progressing toward the exit. By gradually introducing new enemy types, interaction rules, and maze configurations, **Maze of Malice** aims to provide a balanced difficulty progression that remains engaging while maintaining clear and predictable gameplay behavior.

The primary objective of this project is to apply fundamental programming concepts, including data structures, algorithms, and control flow, to the design and implementation of a complete and functional game using the C/C++ programming language, in compliance with the course requirements.

Through this project, students gain practical experience in structuring a program, managing game states, implementing rule-based logic, and integrating algorithmic solutions - such as heuristic enemy pursuit and turn-based state management - within an interactive application. Overall, this work serves as a comprehensive exercise in translating theoretical programming knowledge into a cohesive and playable software product.

TABLE OF CONTENTS

INTRODUCTION i
TABLE OF CONTENTS ii
TABLE OF FIGURES v
1. GROUP INFORMATION 1
1.1. Group Members 1
1.2. Task Assignment & Completion Progress (%) 1
2. ALGORITHMS RESEARCHED AND IMPLEMENTED 2
2.1 Main Algorithms 2
2.2. Maze representation with Collision detector 2
2.2.1. Tile Grid World Model 2
2.2.2. Bitmask Wall Encoding 3
2.2.3. Movement Legality via Bitwise AND 3
2.3. Turn-based execution model 3
2.3.1. Turn Commitment Rule 3
2.3.2. Sequential Update Pipeline 3
2.3.3. Speed Disparity as Difficulty Lever 3
2.4. Enemy AI algorithms and difficulty design 4
2.4.1. Manhattan Distance Heuristic 4
2.4.2. Random Method for Beginner-Friendly Behavior (Introductory Stage) 4
2.4.3. Greedy Local Optimization 4
2.4.4. Enemy Turn Execution (Multi-step and Single-step) 4
2.5. Collision rules and Terminal conditions 4
2.5.1. Trap and Enemy capture 4
2.5.2. Portal (Exit gate) win condition 5
2.6. Undo and Reset by Snapshot History 5
2.6.1. Snapshot-Based State Storage 5
2.6.2. Undo Operation 5
2.6.3. Reset Operation 5
2.7. Pseudocode Algorithm 5
2.7.1. Player Movement Validation with Bitmask Walls 5
2.7.2. Greedy Enemy Step using Manhattan Distance 6
2.7.3. Enemy Turn with Step Budget (Goblin = 2, Slime = 1) 6

2.7.4. Undo / Reset via Snapshot History 7
2.8. Complexity Analysis 7
2.9. Step-by-Step Running Example (One Turn, One Table) 7
3. DETAILED ARCHITECTURE OF THE GAME 8
3.0. Game Overview 8
3.1. Game Interface Overview 8
3.1.1. Main Menu Interface 9
3.1.2. In-game Interface 9
3.2. Maze Design 11
3.2.1. Basic Implementation 12
3.2.2. Maze Difficulty Progression 12
3.2.2.1. 6×6 Maze 12
3.2.2.2. 8×8 Maze 13
3.2.2.3. 10×10 Maze 13
3.3. Character Types 13
3.3.1. Player (Adventurer) 13
3.3.2. Goblin 14
3.3.3. Slime 15
3.4. Control Scheme (Player Input Handling) 15
3.5. Enemy Interaction and Collision Behavior 15
3.5.1. Player - Enemy Collision Rule 15
3.5.2. Slime Interaction (8×8 Maze) 16
3.5.3. Goblin - Goblin Interaction (10×10 Maze) 16
3.5.4. Trap Interaction Rules 16
3.6. Game Scenarios 16
3.7. Game Features 17
3.7.1. Basic Features Implemented 17
3.7.2. Advanced Features 17
3.7.3. Level Structure and Progression 18
4. PRESENTATION OF THE GAME DEVELOPMENT PROCESS 18
4.1. Development Methodology 18
4.1.1. Idea Discussion and Requirement Analysis 18
4.1.2. Research on Maze Games and Algorithms 19
4.1.3. Implementation of Core Game Mechanics 19

4.1.4. Tools and Media Asset Integration 19
4.1.5. Testing and Debugging 20
4.2. Development Timeline 20
5. LIMITATIONS AND FUTURE DEVELOPMENT 21
5.1. Current Limitations 21
5.2. Future Improvements 21
6. CONCLUSION 22
7. ASSETS 23
8. REFERENCES 23

TABLE OF FIGURES

Figure 1. Main Menu Interface 9
Figure 2. SFML Graphic Interface 10
Figure 3. ASCII Graphic Interface 10
Figure 4. Exit cell and Obstacle cell (Trap) 11
Figure 5. Maze grid 11
Figure 6. The Explorer character shown from four directions (front, left, right, back). 13
Figure 7. The Goblin character shown from four directions (front, left, right, back). 14
Figure 8. The Slime character shown from four directions (front, left, right, back), ... 15
inspired by Rimuru Tempest. 15
Figure 9. Flowchart of the Game 17

1. GROUP INFORMATION

1.1. Group Members

No.	Student ID	Full name	Mail
1	25127012	Dương Trung Anh	dtanh2533@clc.fitus.edu.vn
2	25127013	Hà Trần Bội Anh	htbanh2527@clc.fitus.edu.vn
3	25127101	Nguyễn Tuấn Nghĩa	ntnghia2511@clc.fitus.edu.vn
4	25127131	Nguyễn Nhật Quỳnh	nnquynhn2518@clc.fitus.edu.vn
5	25127431	Trần Ngọc Nghĩa	tnnghia2529@clc.fitus.edu.vn

1.2. Task Assignment & Completion Progress (%)

Member	Role	Task Assignment	Progress
Dương Trung Anh	Leader, Backend Developer, Quality Assurance	Project Leadership Backend game logic Algorithm implementation (pathfinding, collision detection) Quality assurance	100%
Hà Trần Bội Anh	Designer, Document writer	Game concept design Dungeon theme UI design Report writing, and documentation	100%
Nguyễn Tuấn Nghĩa	Frontend Developer, Designer	Frontend development Player interface implementation Visual design, and rendering	100%
Nguyễn Nhật Quỳnh	Frontend Developer, Tester	Frontend development Gameplay testing UI testing Bug reporting	100%
Trần Ngọc Nghĩa	Backend Developer, Tester	Backend development Enemy AI, pathfinding logic System testing	100%

2. ALGORITHMS RESEARCHED AND IMPLEMENTED

This section describes the computational logic that governs Maze of Malice, including the grid-based model, collision detection, turn-based, state updates, and the enemy AI used to create strategic puzzle-chase gameplay.

2.1 Main Algorithms

The game project Maze of Malice is built around a turn-based, tile-based grid system. To keep the gameplay strategic while still feeling dynamic and responsive, the project implements the following main algorithms:

- **Bitmask-based wall and Collision detection:** Instead of storing walls as new tiles, our game's method uses bitwise operations to determine wall boundaries within a single integer per grid cell.
- **Turn-based game loop:** A single player action (move or does not move) advances the game by exactly one turn, then triggers enemy updates in a fixed sequence.
- **Greedy enemy AI with Manhattan distance (Local optimization):** The enemy AI utilizes a greedy algorithm based on Manhattan distance to find the closest position toward the player in each turn. Unlike global pathfinding algorithms (such as BFS or Dijkstra), this method makes locally optimal decisions without maintaining a history of visited nodes, simulating a chasing behavior that is predictable yet susceptible to trapping, a desirable trait for puzzle-maze games.
- **State stack management:** A Last-In-First-Out (LIFO) structure is employed to manage game states, enabling "Undo" and "Reset" functionality by storing values of entity coordinates after every valid move.
- **Finite State Machine (FSM) for Scene and Screen management:** The application uses a state-based control flow (MENU <-> PLAYING <-> WIN/LOSE/NEXT LEVEL) with fade transitions to keep UX consistent and maintainable, while creating smooth transitions.

2.2. Maze representation with Collision detector

2.2.1. Tile Grid World Model

The maze is represented as a 2D grid of tiles, and all entities (Player, Goblin, Slime, Trap, Portal) occupy discrete tile coordinates. This model matches the user guide: the player moves exactly one tile per turn, and enemies are also defined by tile-step motion rather than continuous physics. Using this representation ensures that collision and interaction rules are unambiguous: two entities "collide" if and only if they share the same tile coordinate.

2.2.2. Bitmask Wall Encoding

To describe maze topology efficiently, each grid cell stores wall boundaries using a bitmask integer. Each cardinal direction corresponds to a bit (LEFT, RIGHT, UP, DOWN). During movement, checking whether a direction is blocked becomes a constant-time bitwise AND test. The academic value of this design is that it compresses multiple wall configurations into a single primitive value per cell and avoids complicated data structures for a student-scale project.

2.2.3. Movement Legality via Bitwise AND

When an entity attempts to move, the logic reads the current cell's wall mask and tests whether the bit for the intended direction is set. If it is set, movement is rejected and the entity remains on its current tile. This same mechanism is reused across the entire logic layer, keeping the implementation consistent and reducing error-prone duplicated conditions. As a general software-engineering practice, modularizing repeated logic into small reusable functions improves correctness and makes the codebase easier to maintain and review.

2.3. Turn-based execution model

2.3.1. Turn Commitment Rule

Although rendering runs continuously, the game state advances only when a player action constitutes a valid turn. The intended player interactions are explicitly defined in the project guide: movement is one tile per turn, and tactical actions such as Undo/Reset are integrated into the same loop.A key property is that invalid movement (blocked by walls) does not advance the turn, preventing accidental "wasted turns" from unintended key presses.

2.3.2. Sequential Update Pipeline

Once a valid player action is committed, the engine updates the state in a strict order: player movement is applied first, then interactions are evaluated (trap / collision / reaching stair), then enemies perform their moves, and interactions are evaluated again. This deterministic pipeline ensures fairness and supports strategic play: the player can reason about outcomes because the same input in the same state always yields the same result.

2.3.3. Speed Disparity as Difficulty Lever

Maze of Malice intentionally creates difficulty not by complex AI, but by step budget per turn. Goblins are designed as the aggressive threat that moves two tiles per turn, whereas Slimes move one tile per turn, producing different pressure patterns and requiring different lead-distance calculations.

2.4. Enemy AI algorithms and difficulty design

2.4.1. Manhattan Distance Heuristic

Enemy pursuit uses Manhattan distance because movement is restricted to four directions, and Manhattan distance directly measures the number of tile steps required on an open grid. This aligns with the project's stated design that enemy AI is based on "Manhattan distance logic."

2.4.2. Random Method for Beginner-Friendly Behavior (Introductory Stage)

To ensure the first stages are accessible, the project also incorporates a random movement method for the enemy in the introductory maze configuration (6×6). Instead of always selecting the best chasing move, the enemy may choose its next move randomly among valid neighboring tiles (directions not blocked by walls). This stochastic behavior intentionally reduces early punishment and gives beginners time to learn maze navigation, wall constraints, and turn order without facing a perfectly optimized pursuer.

From a gameplay design perspective, controlled randomness creates two benefits: it lowers frustration for new players and increases replayability in small maps because enemy trajectories vary across runs. In implementation, randomness is constrained to legal moves so it never violates collision rules or maze boundaries, and it can be tuned (e.g., avoiding immediate reversal unless necessary) to prevent unnatural oscillation.

2.4.3. Greedy Local Optimization

Instead of computing a full path (e.g., BFS/Dijkstra), enemies evaluate only immediate neighbors and choose a move that reduces Manhattan distance. This greedy approach is intentionally chosen for puzzle gameplay: it creates a readable chase behavior while preserving exploitable weaknesses. In maze environments, the globally correct route may require temporarily increasing distance; greedy agents often refuse such detours and can stall near concave walls, enabling the player to bait and trap enemies through positioning.

2.4.4. Enemy Turn Execution (Multi-step and Single-step)

The same greedy rule is reused across enemy types, but applied with different step counts. Goblins apply the greedy movement twice per turn, while Slimes apply it once, which is consistent with the intended gameplay identity described in the user guide. This design keeps the AI explainable in a report while still producing clear difficulty progression.

2.5. Collision rules and Terminal conditions

2.5.1. Trap and Enemy capture

The game defines loss conditions as tile-overlap events. Stepping on a Trap is an instant Game Over, and being caught by an enemy (enemy entering the player's tile) also ends

the run. Centralizing these checks prevents state inconsistencies and ensures the rendered state corresponds to a single authoritative logical state.

2.5.2. Portal (Exit gate) win condition

A level is cleared when the player reaches the Stair/exit tile, which is the destination object defined in the project guide. The win check is performed deterministically within the same turn pipeline so that victory is consistent and predictable.

2.6. Undo and Reset by Snapshot History

2.6.1. Snapshot-Based State Storage

Undo and Reset are implemented by storing a history of snapshots that minimally capture deterministic gameplay state (primarily entity coordinates). Because the game is tile-based and turn-based, coordinates are sufficient to restore past states reliably. This approach also reflects good engineering practice: instead of attempting to "reverse" logic procedurally, the system restores a previously recorded state, which reduces corner-case bugs.

2.6.2. Undo Operation

Undo reverts the most recent committed move by restoring the previous snapshot. The feature is exposed as both a hotkey (press 'R' or designated button) and UI control in the guide, reinforcing its role as a core mechanic rather than a debugging tool.

2.6.3. Reset Operation

Reset restores the level's initial snapshot and clears the intermediate history so that the new attempt starts cleanly. This feature is similarly exposed (press 'M' or designated button) and supports repeated planning attempts, which is essential for puzzle-focused levels.

2.7. Pseudocode Algorithm

2.7.1. Player Movement Validation with Bitmask Walls

```
FUNCTION PlayerMove(wallGrid, key, playerPos):
  DIR_BIT('LEFT ARROW') = 1   // LEFT
  DIR_BIT('d') = 2   // RIGHT
  DIR_BIT('w') = 4   // UP
  DIR_BIT('s') = 8   // DOWN

  IF key NOT IN {'w','a','s','d'}:
    RETURN (playerPos, VALID=false)
```

```
    mask = wallGrid[playerPos.r][playerPos.c]
    bit  = DIR_BIT(key)

    IF (mask AND bit) != 0:
        RETURN (playerPos, VALID=false)   // blocked by wall

    nextPos = Neighbor(playerPos, key)
    RETURN (nextPos, VALID=true)
END FUNCTION
```

2.7.2. Greedy Enemy Step using Manhattan Distance

```
FUNCTION GreedyStep(wallGrid, enemyPos, playerPos):
    bestDir  = NONE
    bestDist = Manhattan(enemyPos, playerPos)

    FOR dir IN [LEFT, RIGHT, UP, DOWN]:
        bit  = BitOf(dir)              // LEFT=1, RIGHT=2, UP=4, DOWN=8
        mask = wallGrid[enemyPos.r][enemyPos.c]

        IF (mask AND bit) != 0:
            CONTINUE                   // blocked

        cand = Neighbor(enemyPos, dir)
        d = Manhattan(cand, playerPos)

        IF d < bestDist:
            bestDist = d
            bestDir  = dir
    END FOR

    IF bestDir IS NONE:
        RETURN enemyPos                // local minimum / cannot improve
    RETURN Neighbor(enemyPos, bestDir)
END FUNCTION
```

2.7.3. Enemy Turn with Step Budget (Goblin = 2, Slime = 1)

```
FUNCTION EnemyTurn(wallGrid, enemyPos, playerPos, stepsAllowed):
    FOR i FROM 1 TO stepsAllowed:
        enemyPos = GreedyStep(wallGrid, enemyPos, playerPos)
```

```
    IF enemyPos == playerPos:
      RETURN (enemyPos, CAUGHT=true)
  END FOR
  RETURN (enemyPos, CAUGHT=false)
END FUNCTION
```

2.7.4. Undo / Reset via Snapshot History

```
DATA:
  history = list of snapshots
  // snapshot stores: playerPos, goblinPos, goblin2Pos, slimePos, ...

FUNCTION SaveSnapshot(state):
  history.push_back(Clone(state))

FUNCTION Undo():
  IF history.size <= 1: RETURN history.back()
  history.pop_back()
  RETURN history.back()

FUNCTION Reset():
  initial = history.front()
  history.clear()
  history.push_back(initial)
  RETURN initial
```

2.8. Complexity Analysis

The project's core algorithms are intentionally lightweight. A single wall check is a bitwise AND and runs in O(1) time. For enemy movement, each greedy step evaluates at most four neighbors and also runs in O(1) time. Therefore, the per-turn complexity is:

$$O(enemies \times stepsAllowed)$$

Since both the number of enemies and their step budgets are small fixed constants in our level design, gameplay updates are effectively constant-time per turn, supporting smooth real-time rendering even on modest machines.

2.9. Step-by-Step Running Example (One Turn, One Table)

Consider an open corridor in a 6×6 grid (no walls blocking left movement).
Player: (player_r, player_c) = (1, 1)
Goblin: (goblin_r, goblin_c) = (1, 4)
Goblin moves two greedy steps in one player turn.

Step 1:

(Goblin at (1,4)): Manhattan distance to player = |1-1| + |4-1| = 3

Candidate move	New position	New distance	Selected?
LEFT	(1,3)	2	✓ (strictly smaller)
RIGHT	(1,5)	4	✗
UP	(0,4)	4	✗
DOWN	(2,4)	4	✗

Goblin moves to (1,3).

Step 2:

(Goblin at (1,3)): distance = 2 → Goblin again selects LEFT to (1,2) (distance becomes 1).

If a wall blocked LEFT at (1,4), then the wall bit would prevent that direction from being considered legal; if no other legal direction strictly decreases distance, the Goblin would remain stationary for that step—illustrating how maze geometry can create "traps" against greedy pursuit.

3. DETAILED ARCHITECTURE OF THE GAME

3.0. Game Overview

Maze of Malice is a 2D dungeon-themed puzzle game inspired by the classic Mummy Maze. The player takes on the role of an adventurer attempting to escape from a series of mazes while avoiding enemies and environmental hazards. The game employs a turn-based movement system, emphasizing strategic decision-making and logical planning. Each level gradually increases in complexity by introducing new maze layouts, enemy types, and interaction rules, providing a balanced difficulty progression.

This section describes the detailed architecture of **Maze of Malice**, including game interface, maze design, character types, control schemes, enemy interactions, and other features that constitute the gameplay experience.

3.1. Game Interface Overview

3.1.1. Main Menu Interface

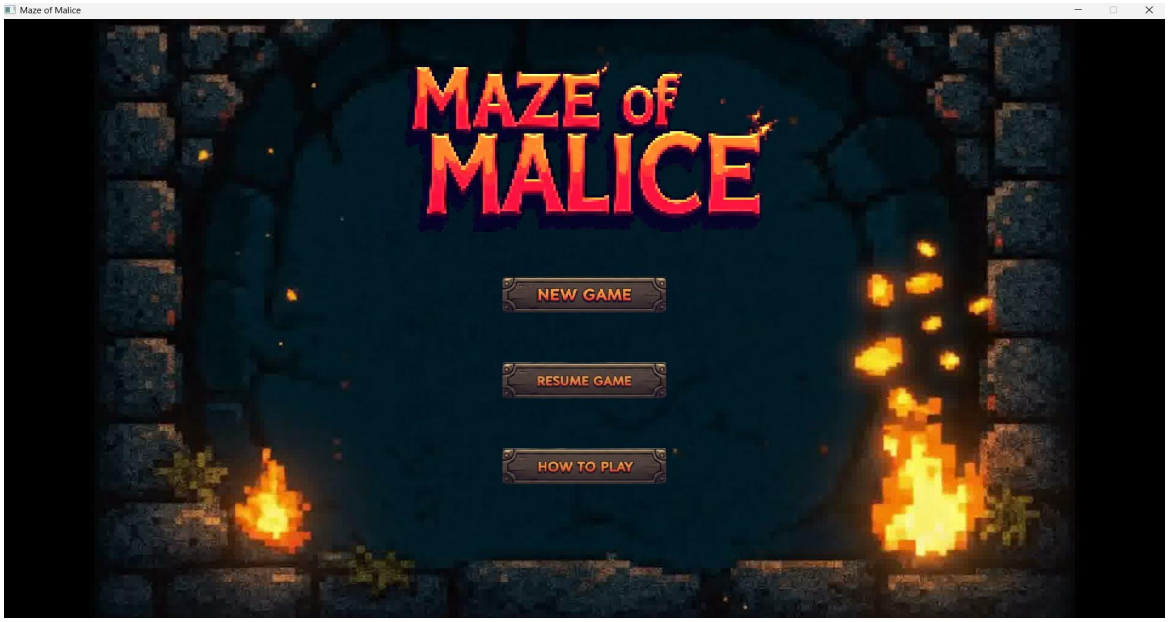

Figure 1. Main Menu Interface

The Main Menu serves as the first point of interaction for the player, providing access to the core game functionalities before entering the maze. It is designed to be simple, intuitive, and visually consistent with the overall game theme. The main components of the menu include:

1. **New game:** Initiates a new game session, loading the first level of the maze.
2. **Resume game:** Allows the player to resume from the last saved progress, supporting continued gameplay.
3. **How to Play:** Provides instructions on game rules, controls, and objectives to guide new players.

The menu layout is designed for easy navigation using keyboard or mouse input. Visual feedback is provided when hovering over or selecting buttons, enhancing the user experience. The menu also features thematic graphics and background music that reflect the dungeon-adventure style of the game, helping to immerse players even before gameplay begins.

3.1.2. In-game Interface

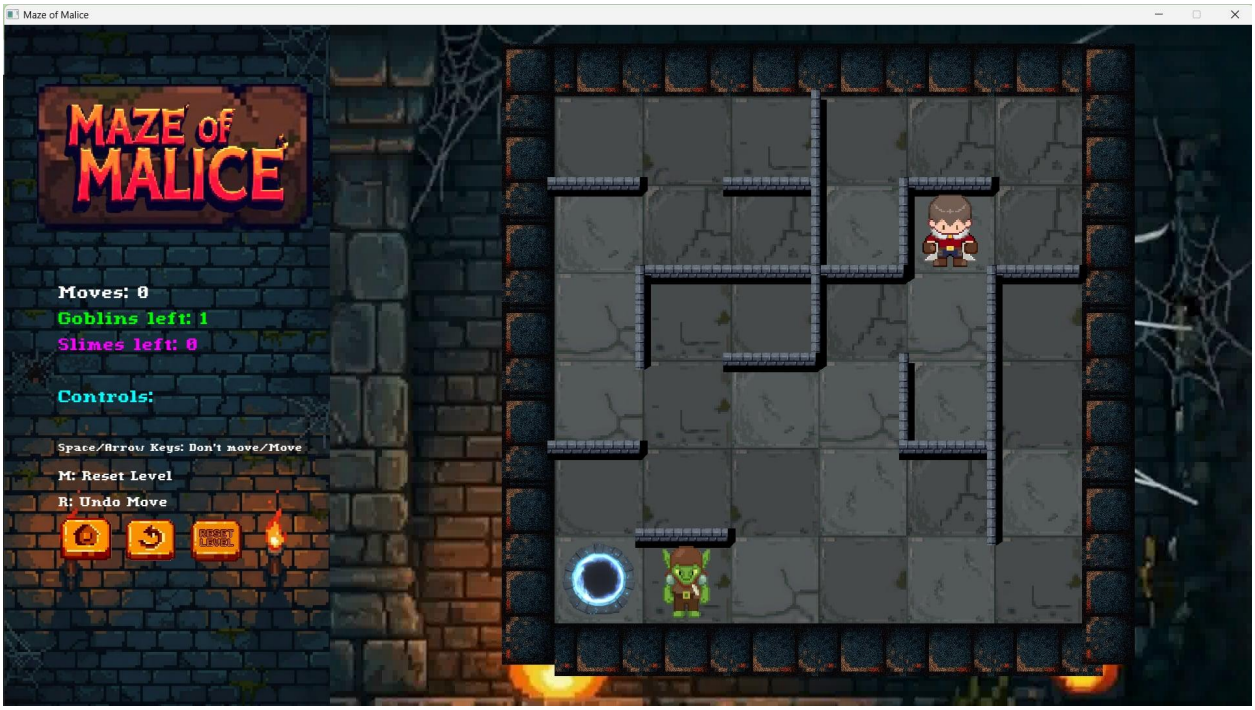

Figure 2. SFML Graphic Interface

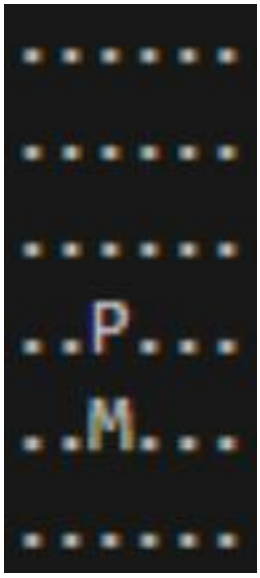

Figure 3. ASCII Graphic Interface

The game is presented in a 2D top-down view, where the entire maze is displayed on the console screen using ASCII graphics and SFML graphics, in accordance with the project's basic graphic requirements.
The dungeon theme is conveyed through the maze layout's arrangement, which features stone-like walls and narrow corridors, creating a dark and mysterious atmosphere despite the use of text-based graphics.

The main game interface consists of the following components:

- **Maze grid**: Represents the structure of the dungeon, including walls and walkable paths.
- **Player character**: Displayed at the current position of the explorer within the maze.
- **Enemy characters**: Represent hostile entities that move inside the maze and attempt to catch the player.
- **Exit cell and obstacles**: Indicate the goal of the level and elements that block movement.

This interface allows players to clearly observe the game state and make movement decisions turn by turn.

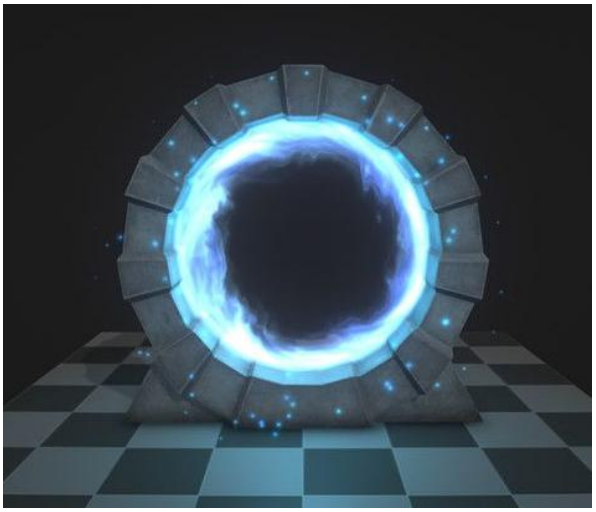

Figure 4. Exit cell and Obstacle cell (Trap)

(Credit: Pinterest.com) *(Credit: X.com)*

3.2. Maze Design

Figure 5. Maze grid

The maze is designed using a grid-based structure, where each cell represents a walkable tile rather than a wall.

The player and enemies move from one cell to an adjacent cell, unless a wall blocks the movement.

Unlike traditional tile-based mazes, where walls occupy entire cells, walls in this game are placed along the boundaries between cells, acting as barriers that prevent movement between two adjacent tiles.

This design allows for more flexible maze layouts while maintaining a clear and consistent movement system.

3.2.1. Basic Implementation

To satisfy the project requirements, the maze design follows these rules:

- **Fixed maze sizes:** The game includes three predefined maze dimensions: 6×6, 8×8, and 10×10, representing different difficulty levels.
- **Pre-designed layouts:** All maze layouts are manually designed. Each layout is carefully constructed to control difficulty and ensure proper gameplay balance. One thing is randomized is the layout of the grid, the game uses a random method to draw each tile.
- **Wall placement between cells:** Walls are located at the intersections between adjacent cells, blocking movement in specific directions rather than occupying entire cells. This approach clearly defines valid and invalid movements while keeping all cells as potential paths.
- **Inner wall limitation:** The total number of internal walls does not exceed 50% of the possible wall positions, preventing overly restrictive mazes.
- **Guaranteed valid path:** Every maze is verified to ensure that at least one valid path exists from the Explorer's starting position to the exit.

3.2.2. Maze Difficulty Progression

Maze difficulty increases progressively as the grid size expands from **6×6 to 10×10**, not only through maze complexity but also through the introduction of additional enemies and special tiles.

3.2.2.1. 6×6 Maze

This level is designed as an introductory stage. The maze size is intentionally small so that new players can focus on learning the core mechanics: tile-based movement, wall constraints, and how turn-based updates work. At this stage, the game includes only one Goblin, ensuring the player experiences enemy pressure without being overwhelmed.

To keep the first stage beginner-friendly, the Goblin's behavior in the 6×6 maze incorporates a random method (stochastic movement). Instead of always choosing the mathematically best chasing move every time, the Goblin selects its next move randomly among the valid neighboring tiles (i.e., directions that are not blocked by walls). When several valid directions exist, the selection is uniform so the Goblin's motion appears less "perfect" and less punishing. To prevent unnatural back-and-forth oscillation, the random method may also prioritize directions that do not immediately reverse the previous step unless no other legal option exists.

This controlled randomness has two benefits for an introductory level. First, it reduces early frustration by giving players more time to understand the rules and map layout. Second, it increases replayability even in a small maze, because the Goblin's exact trajectory is not identical in every run, encouraging players to practice planning and positioning rather than memorizing a single fixed chase pattern.

3.2.2.2. 8×8 Maze

The difficulty increases with a larger maze size and the addition of more gameplay elements.

This level includes **one Goblin, one Slime, and trap cells**, requiring players to plan their movements carefully while avoiding both enemies and hazardous tiles.

3.2.2.3. 10×10 Maze

This level presents the highest difficulty.

It features **two Goblin enemies and trap cells**, combined with a larger maze area and more complex wall layouts, significantly increasing the challenge and strategic depth.

By gradually increasing maze size, enemy count, and environmental hazards, the game provides a smooth difficulty curve while maintaining fairness and playability.

3.3. Character Types

3.3.1. Player (Adventurer)

Figure 6. The Explorer character shown from four directions (front, left, right, back).

The player controls an adventurer who is trapped inside a dungeon, with the main objective of escaping the maze by reaching the exit without being caught by enemies.

The adventurer can move one cell per turn in one of the four cardinal directions (up, down, left, or right), provided that the movement is not blocked by a wall.

Each action performed by the player corresponds to a single step, which allows the game to operate in a turn-based manner and encourages strategic planning.

3.3.2. Goblin

Figure 7. The Goblin character shown from four directions (front, left, right, back).

(Reference: freepik.com)

The Goblin is the primary enemy introduced from the early stages of the game and acts as the main source of pressure on the player. In our implementation, the Goblin's movement is driven by a greedy pursuit heuristic based on Manhattan distance. Instead of computing a full shortest path (e.g., BFS/Dijkstra), the Goblin evaluates its immediate neighboring tiles each step and selects a move that strictly reduces the Manhattan distance to the player whenever such a move exists.

This design choice is intentional for a puzzle-maze game: the Goblin appears "intelligent" and actively chasing, but it remains predictable and exploitable because greedy pursuit can stall near certain wall shapes where the globally correct route would require temporarily increasing distance. As a result, the player can still win by strategic positioning and planning rather than being forced to outrun perfect pathfinding.

To scale difficulty without changing the AI logic itself, the Goblin is configured as a faster threat: it can move two tiles per turn (two greedy steps), which increases urgency and reduces the player's margin for error. In higher stages, the game may also spawn multiple Goblins simultaneously, increasing spatial complexity while preserving consistent, understandable behavior.

3.3.3. Slime

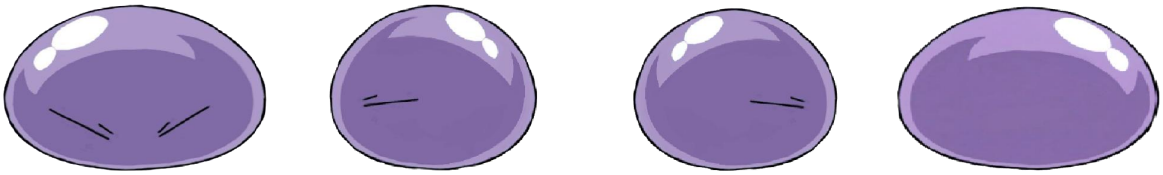

Figure 8. The Slime character shown from four directions (front, left, right, back), inspired by Rimuru Tempest.

Slime is an additional enemy type introduced in later levels to further increase gameplay difficulty.

Unlike the Goblin, the Slime moves only one cell per turn, making it slower; however, its primary role is not speed but effective area control within the maze.
By occupying key positions within the maze, the Slime restricts available paths and forces the player to make more strategic movement decisions when navigating the environment.

3.4. Control Scheme (Player Input Handling)

The game uses a keyboard-based control scheme designed for a console environment.

The player can control the adventurer using the following keys:

- **Arrow keys**: Move the adventurer one cell in the corresponding direction (up, down, left, right), provided the movement is not blocked by a wall.
- **Space:** Skip the player's turn without moving the adventurer; during this turn, enemy characters are still allowed to move according to their behavior rules.
- **R**: Undo the previous move, allowing the player to revert one step.
- **M**: Reset the current level and restart the maze from the initial state.
- **Q**: Exit the game immediately.

This control scheme ensures simplicity, responsiveness, and ease of use, allowing players to focus on puzzle-solving and strategic movement.

3.5. Enemy Interaction and Collision Behavior

3.5.1. Player - Enemy Collision Rule

In the game, collisions between the player and enemies are handled uniformly regardless of enemy type.

Whenever the player collides with an enemy, including both Goblin and Slime, the player immediately loses the game.

Collision detection is evaluated after each movement turn to ensure consistent and predictable gameplay behavior.

3.5.2. Slime Interaction (8×8 Maze)

In the 8×8 maze, the Slime is introduced as an additional enemy to increase the overall difficulty of the level.

Similar to the Goblin, the Slime is classified as a hostile entity, and any collision between the player and a Slime results in an immediate loss.

Enemy–enemy interactions in this maze are asymmetric. When a Goblin collides with a Slime, the Slime is removed from the game, while the Goblin remains active and continues its movement normally.

This interaction rule adds strategic depth by allowing dynamic changes in enemy distribution while preserving clear and predictable gameplay behavior for the player. Furthermore, it prevents excessive enemy congestion while reinforcing the Goblin's dominant role as the primary threat.

3.5.3. Goblin - Goblin Interaction (10×10 Maze)

In the 10×10 maze, two Goblin enemies are present simultaneously, which introduces the possibility of enemy–enemy collisions.

When both Goblins move into the same cell, a collision occurs, causing one Goblin to be removed from the game while the remaining Goblin continues to function normally.

This interaction rule helps prevent excessive difficulty caused by overlapping enemies and contributes to smoother and more balanced gameplay in larger maze layouts.

3.5.4. Trap Interaction Rules

Trap cells are introduced in higher-level mazes as environmental hazards designed to increase gameplay difficulty.

When the player moves onto a trap cell, the game immediately ends, resulting in a loss.

In contrast, enemy entities, including both Goblins and Slimes, are not affected by trap cells and are able to move through them without any consequences.

This design choice increases the challenge by limiting the player's safe movement options while ensuring that enemy behavior remains predictable and consistent across all levels.

3.6. Game Scenarios

At the beginning of each level, the player is placed at a predefined starting position within the maze, while enemy entities are spawned at fixed locations.

The game proceeds in a turn-based manner, where:

1. The player performs an action.
2. Enemy movements are processed according to their behavior rules.
3. Collision and interaction checks are performed.

A level ends when one of the following conditions is met.

1. The player wins the level by successfully reaching the exit cell
2. The player loses the game if a collision with an enemy occurs or if the player steps onto a trap cell.

This structured gameplay flow ensures consistency across different maze sizes, while different levels apply varying configurations of enemies and environmental hazards, resulting in a gradual increase in gameplay difficulty as outlined in the maze difficulty progression.

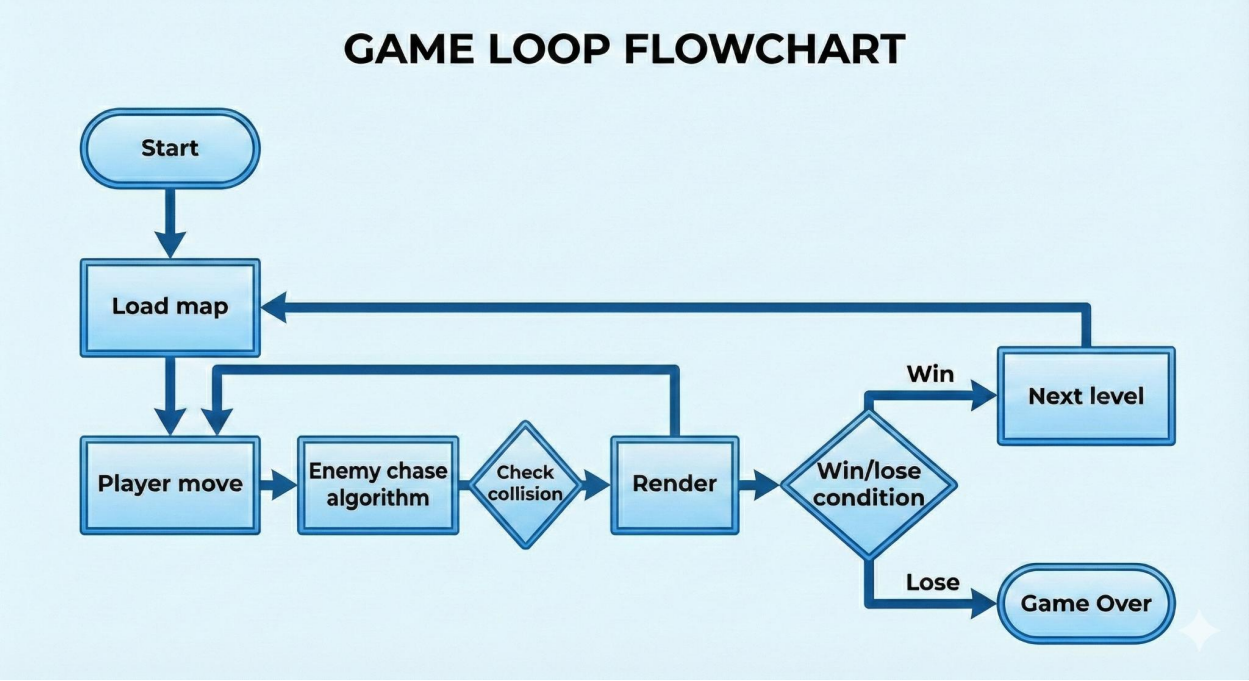

Figure 9. Flowchart of the Game

3.7. Game Features

3.7.1. Basic Features Implemented

The following basic features have been implemented to ensure core gameplay functionality, usability, and player convenience:

- **Undo last move**: Allows the player to revert the most recent movement, supporting trial-and-error gameplay and encouraging strategic planning.
- **Reset level**: Enables the player to restart the current level from its initial state at any time.
- **Background sound**: Provides ambient audio effects to enhance the overall dungeon atmosphere.
- **Multiple difficulty levels**: The game includes multiple levels with increasing difficulty based on maze size, enemy behavior, and gameplay complexity.

3.7.2. Advanced Features

In addition to the basic requirements, several advanced features have been implemented to further enhance gameplay depth and player experience:

- **Automatic save system**: The game automatically saves the current game state, allowing players to resume progress without manual intervention.
- **Progressive difficulty scaling**: Game difficulty increases gradually through larger maze layouts, additional enemy types, and environmental hazards.
- **Trap cells**: Trap tiles are introduced in higher levels, causing the player to lose immediately upon contact.
- **Multiple enemy instances**: Higher-level mazes include two or more enemy characters simultaneously, increasing strategic and spatial complexity.
- **Additional sound effects**: Extra audio effects are provided for player movement, level completion, and level failure events to improve feedback and immersion.

3.7.3. Level Structure and Progression

The game consists of **five stages**, including an introductory warm-up stage followed by four main gameplay levels.

The warm-up stage is designed to help players become familiar with the basic controls and gameplay mechanics before entering the main levels.

- **Warm-up Stage**: A simple introductory stage that allows the player to practice movement and understand basic game rules in a low-pressure environment.
- **Level 1 and Level 2**: These levels use a **6×6 maze** with **one Goblin enemy**, serving as early gameplay stages that reinforce core mechanics and enemy avoidance skills.
- **Level 3**: This level uses an **8×8 maze** and introduces **one Goblin, one Slime, and trap cells**, increasing difficulty by adding new enemy interactions and environmental hazards.
- **Level 4**: The final level uses a **10×10 maze** with **two Goblin enemies and trap cells**, representing the highest difficulty and requiring careful strategic movement.

This progression provides a smooth learning curve, allowing players to gradually adapt to new mechanics while maintaining consistent gameplay rules.

4. PRESENTATION OF THE GAME DEVELOPMENT PROCESS

4.1. Development Methodology

The development of the game followed an iterative and incremental approach, enabling the team to progressively build, test, and refine the game.
This approach allowed for continuous feedback, early detection of issues, and improvement of gameplay features across multiple development phases.

4.1.1. Idea Discussion and Requirement Analysis

The team discussed project requirements, selected a dungeon theme, and defined core gameplay mechanics inspired by the classic *Mummy Maze*. Roles were assigned according to each member's expertise, covering frontend, backend, design, testing, and documentation.

4.1.2. Research on Maze Games and Algorithms

During the research phase, the team analyzed classic maze-chase and puzzle-maze designs (including Mummy Maze–style mechanics) to identify an enemy behavior model that is both challenging and suitable for turn-based strategy. Several approaches were considered, including global pathfinding methods (e.g., BFS/Dijkstra) and heuristic-based pursuit.

For the current version of Maze of Malice, we selected a greedy Manhattan-distance pursuit algorithm as the core enemy AI. This approach provides strong chase pressure while remaining readable and strategically fair: enemies make locally optimal decisions without calculating an entire route, allowing maze geometry to play a meaningful role. In particular, greedy pursuit can be influenced by walls and corners, enabling the player to bait enemies, create spacing, and solve levels using planning rather than relying on reflexes.

To achieve difficulty progression, the project increases challenge primarily through level layout complexity, additional obstacles (traps), and enemy configuration (e.g., Goblin moving two steps per turn, Slime moving one step per turn). More advanced global pathfinding (such as BFS-based shortest-path chasing) is considered a potential future extension if the project later adds a separate "hard mode" AI variant.

4.1.3. Implementation of Core Game Mechanics

- **Programming Language:** C++
- **Graphics & Audio Library:** ASCII Graphics & SFML (Simple and Fast Multimedia Library)

Core mechanics implemented included:

- Grid-based movement and turn-based player/enemy movement.
- Collision detection and interaction rules supporting traps and multiple enemy types.

4.1.4. Tools and Media Asset Integration

The following tools and platforms were employed to create and manage media assets efficiently:

- **Graphics & Illustration:** Canva, Gemini.ai, ibisPaint X
- **Video & Cutscenes:** Pixverse.ai
- **Image Editing:** ezremove.ai for background removal

- **Royalty-Free Assets:** Pixabay.com for images and audio clips
- **Documentation & Content Drafting:** ChatGPT, Gemini
- **Media Conversion & Processing:** CloudConvert and FFmpeg

These tools allowed the team to integrate high-quality visual and audio assets seamlessly into the game.

4.1.5. Testing and Debugging

Testing was conducted iteratively, with QA team members verifying game logic, collision detection, and feature functionality. Bugs were logged and fixed progressively, ensuring stability and a smooth gameplay experience.

This structured methodology, combining programming, algorithm implementation, and media integration, enabled the team to deliver a complete and functional 2D dungeon-themed game in compliance with course requirements.

4.2. Development Timeline

The project was conducted over a period of seven weeks, with clearly defined objectives and deliverables for each phase.

Week	Date	Key Tasks & Activities
1	Nov 3 – Nov 9	Task assignment and overall project planning Establishment of basic program structure and visualization approach Playing and analyzing reference maze games Progress reporting every two days
2	Nov 10 – Nov 16	Completion of basic visual assets for the frontend Implementation of core backend logic and data structures Development of basic player movement and grid navigation Daily progress reporting.
3	Nov 17 – Nov 23	Testing and bug detection by QA Integration of visual assets into the frontend Proposal and discussion of advanced gameplay features Bug fixing and logic refinement Daily progress reporting.
4	Nov 24 – Nov 30	Implementation of advanced gameplay features Testing and balancing new features Continuous debugging and optimization Daily progress reporting.

5	Dec 1 – Dec 7	Completion of all planned game features Comprehensive project testing Final bug fixes and performance checks Daily progress reporting.
6	Dec 8 – Dec 14	Recording of gameplay demonstration videos Preparation of project report Final testing and quality assurance checks.
7	Dec 15 – Dec 21	Final revisions of the report Final gameplay verification Overall project completion and submission.

5. LIMITATIONS AND FUTURE DEVELOPMENT

5.1. Current Limitations

The current version of the game, while functional and complete in terms of core gameplay, exhibits several notable limitations. Firstly, all maze levels are predefined and fixed, which reduces replayability as players can memorize paths and optimal strategies after a few plays. Secondly, the game includes a limited number of levels, restricting the variety of challenges and overall playtime, which may affect long-term player engagement. In addition, animations for player movement, enemy behavior, and interactions are relatively basic, making the game visually less dynamic and immersive. Finally, while the graphic assets are functional, they are not fully polished: character sprites, background textures, and environmental elements lack detail and artistic refinement, which reduces the overall visual appeal.

5.2. Future Improvements

To address these limitations and enhance the game experience, several improvements are planned for future development. Implementing an algorithm for random maze generation would increase replayability by providing a fresh challenge for each playthrough. Expanding the number of levels with varied layouts, enemy arrangements, and difficulty progression would enhance long-term engagement and provide a more comprehensive gameplay experience. Additionally, introducing new gameplay elements such as traps, gates, and keys could add strategic depth, requiring players to plan their movements carefully. Improvements in visuals and animations, including more detailed character sprites, smoother enemy movements, and dynamic environmental effects, would enhance immersion. Complementary audio enhancements, such as richer sound effects and background music, would further engage players. Finally, incorporating a login system with player profiles and a leveling mechanism would allow progress tracking and

personalized challenges, supporting long-term engagement and more advanced game features in the future.

6. CONCLUSION

This project successfully demonstrates the application of fundamental programming concepts to the development of **Maze of Malice**, a 2D dungeon-themed puzzle game inspired by the classic *Mummy Maze*. Through the iterative and incremental development methodology, the team implemented core gameplay mechanics, including turn-based movement, collision detection, enemy AI behavior, and level progression. The integration of visual and audio assets using tools such as SFML, Canva, ibisPaint X, and other media platforms enhanced the game's presentation and player experience.

Throughout the seven-week development timeline, the project achieved all planned objectives, including basic and advanced game features, testing and debugging, and comprehensive documentation. The final product provides a functional and engaging game that meets course requirements while illustrating the effective application of algorithms, data structures, and programming logic in a practical context.

Despite certain limitations, such as static maze layouts, a limited number of levels, and simple graphics, **Maze of Malice** establishes a solid foundation for future enhancements. Planned improvements, including random maze generation, additional gameplay elements, richer visuals, and a player account system, provide clear pathways for further development.

In conclusion, this project not only fulfills the academic objectives of the Fundamentals of Programming course but also serves as a practical exercise in translating theoretical programming knowledge into a cohesive, playable software product. It highlights the importance of systematic planning, iterative development, and the integration of multimedia tools in delivering complete and functional game experience.

7. ASSETS

[1] Pinterest, "Pixel art exit door," Pinterest. [Online]. Available: https://de.pinterest.com/pin/599963981589463236/. *[Accessed: 21-Dec-2025]*.

[2] RunninBlood, "Trap design," X (Twitter). [Online]. Available: https://x.com/RunninBlood/status/1889092766728114239.

[3] Freepik, "Pixel art style green goblin character," Freepik. [Online]. Available: https://www.freepik.com/premium-vector/pixel-art-style-green-goblin-character-with-pointed-ears-brown-outfit_285201564.htm.

[4] R. Mikami, "That Time I Got Reincarnated as a Slime," anime series, Eight Bit, 2018.

[5] Pixabay, "Mystery Eerie Music Box with Dark Tones," Pixabay. [Online]. Available: https://pixabay.com/music/mystery-eerie-music-box-with-dark-tones-289437/.

[6] Pixabay, "Concrete Footsteps 1," Pixabay. [Online]. Available: https://pixabay.com/vi/sound-effects/concrete-footsteps-1-6265/.

[7] Pixabay, "Wrong Place," Pixabay. [Online]. Available: https://pixabay.com/sound-effects/wrong-place-129242/.

[8] Pixabay, "Defeat Background," Pixabay. [Online]. Available: https://pixabay.com/vi/sound-effects/defeat-background-39613/.

8. REFERENCES

[1] GeeksforGeeks, "Manhattan distance," GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/data-science/manhattan-distance/.

[2] GeeksforGeeks, "Bitwise Operators in C/C++," GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/cpp/cpp-bitwise-operators/.